Blockchain Data Structures

Introduction

Blockchain is a distributed infrastructure and computing paradigm formed by reorganizing some mature technologies like

- Hash function
- Merkle tree
- Proof of work (PoW)
- Public key encryption,
- Digital signature etc.

Blockchain technology is considered to be an anti-counterfeiting, non-tampering and easy to implement smart contracts, and is known as a new technology that will probably lead to a massive social change.

Blockchain technology uses cryptographic technology to hide user information, while all transaction information is verified and stored by distributed network.

It can be divided into

- Public chain,
- Alliance chain and
- Private chain,

According to various application scenarios its utilized for. Bitcoin, Ethereum etc., are typical public chains. Here, we have no restrictions on the joining and exiting of nodes; data can be easily accessed, which provides an extraordinary opportunity for data analysts to analyze various behaviors in the system by means of transaction data.

Public chains such as Bitcoin, Ethereum etc., have obtained a large number of users and accumulated a large amount of transaction data. In 2020, more than 45 million users worldwide hold Bitcoin.

Today, various industries have introduced blockchain as the underlying technology, which has inevitably lead to the existence of a large amount of information in the form of blockchain data, making the study and research of blockchain data to be important with both theoretical and practical significance.

Blockchain technology is an emerging technology that is currently widely concerned by researchers. Let's look into the three-layer model from the perspective of data analysis. Based on this let's also discuss the analysis of blockchain data structure, data types, and the data structure and operation principle of smart contracts. We shall also cover the seven problems and their significance in blockchain data analysis.

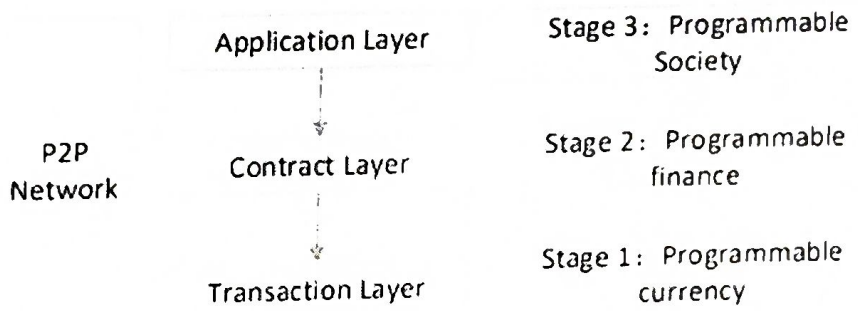
Blockchain Architecture

Blockchain was initiated as the underlying technology of Bitcoin. by "Satoshi Nakamoto" who proposed a digital currency in his whitepaper. Without authoritative intermediaries, people who do not trust can pay directly in Bitcoin. Blockchain Technology was designed as a new application mode of computer technology, using distributed data storage, point-to-point transmission, consensus mechanism and encryption algorithm.

In terms of the technical architecture of the blockchain, a five-layer framework was proposed: data layer, network layer, consensus layer, incentive layer, contract layer and application layer.

From the perspective of privacy protection; another version proposes a three-tier framework: network layer, transaction layer and application layer.

From the perspective of facilitating data analysis, lets see a three-tier framework: application layer, contract layer, transaction layer.



- Transaction layer, correspond to the block chain development stage 1, can be programmed currency. Transaction is not only a means to change blockchain data, but also an important basic data in blockchain. This forms of Recording transactions shall ensure the global uniqueness and irreversible modification of blockchain data.
- Contract layer, corresponding to the phase 2, of the blockchain development: functions as a digital protocol that uses algorithms and procedures to compile various contract terms, which deploys the blockchain model, and that shall be automatically executed according to rules. The smart contract is also an important data type which we call contract data. The deployment and operation of smart contracts is inseparable from transactions; moreover it would generate new transaction data.
- Application layer, phase 3of blockchain development; refers to programmable society, such as decentralized application, decentralized autonomous organization. It can be built on top of the contract layer to implement many complex automation functions. As the blockchain technology is still in its early stages of development, there do exist a lack of application data combined with real scenarios.

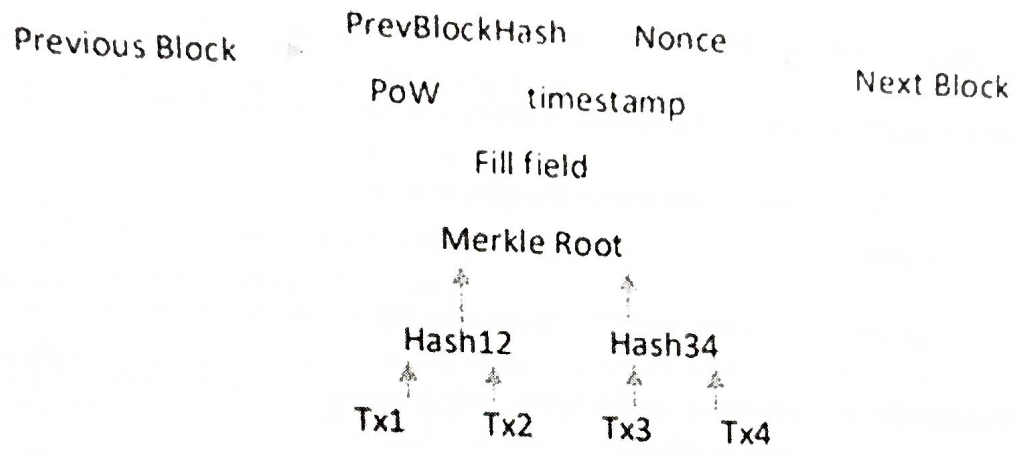
As the functional environment of the blockchain is distributed, this three-tier framework operates in a distributed environment.

Blockchain Data Structure

The blockchain is constructed into a chain structure in units of blocks. Though the data builds on diverse blockchain platforms may be slightly different, but basically the structure is almost same. Considering Bitcoin structure; each block is composed of a block header and a block body.

- Block body stores multiple transactions that occurred after the previous block;
- Block header stores a pre-block hash (PrevBlockHase) , random number (Nonce), Merkle root (MerkleRoot), PoW (Proof of work) etc.

It guarantees irreversible modification based on two hash structures, Merkle tree and block list.



Merkle tree: was first proposed by Ralph Merkle to generate digests of digital certificate directories. Bitcoin uses the simplest binary Merkle tree. Each node in the tree is a hash value, and stores a SHA256 hash value of a transaction data. The values of two leaf nodes are spliced together, and then the values of the parent node are obtained by hashing operation.

Repeatedly calculate the hash value of the parent node until the root is generated, that is, transaction merkle root. With the Merkle root, tampering of any transaction data in the block is detected, ensuring the integrity of the transaction data. Without the involvement of other nodes in the tree, we can confirm whether a transaction occurs based on the SPV (Simplified Payment Verification) only according to the direct branch from the transaction node to the Merkle root path.

For example, only the nodes Hash3, Hash12, and Merkle roots in the above Figure can be used to verify that the transaction Tx4 is located in the block

In Ethereum, the Merkle root is calculated using the Merkle Patricia tree because the transaction data in the block does not change much, but the status data often changes and the number is large. When building a new block, the MerklePatricia tree only needs to calculate the account status that has changed in the new block, and branches with no change in status can be directly referenced without recalculation.

Block list. The hash value of the block header is obtained by performing a SHA256 hash operation on the metadata such as the Prev-BlockHash, the Nonce, and the Merkle root in the block header. PrevBlockHash stores hash value of the previous block. All blocks are linked together in the order of generation with PrevBlockHash as a hash pointer, forming a block list. The block header contains the transaction Merkle root, so you can verify whether the block header and transaction data have been tampered with by Hash operation. The block header also contains the previous block hash value PrevBlockHash, so it is also possible to verify whether all blocks from the previous block of the block to the creation block have been tampered with by block hashing. Relying on the PrevBlockHash, all blocks are interlocked, and any block is tampered with, causing a chain change of all the block hash pointers. When a block and all previous blocks are downloaded from an untrusted node, block hashing can be used to verify whether each block has been modified.

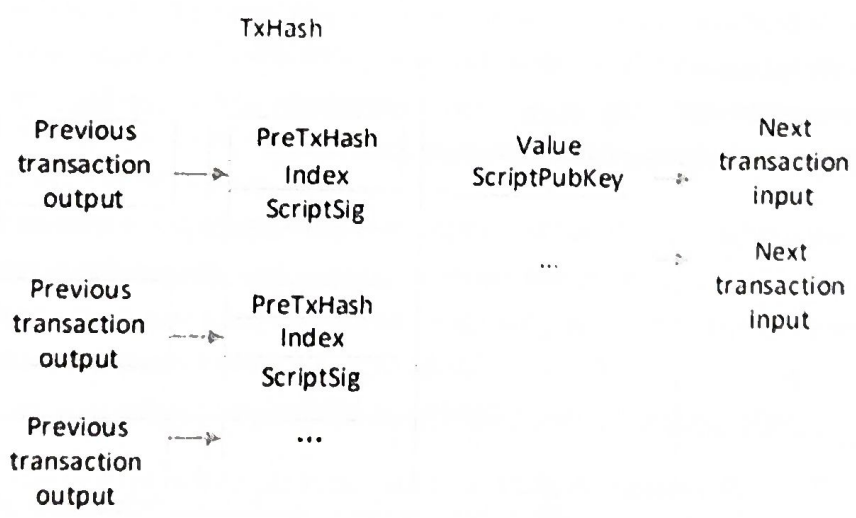
Blockchain Data Types

Two types of record-keeping models are popular in today's blockchain networks.

- Transaction (UTXO- Unspent Transaction Output) Model and
- Account-based Model.

The UTXO model is employed by Bitcoin, and Ethereum uses the Account/Balance Model.

1. Transaction data model. A transaction in the blockchain is usually a transfer, following Figure shows the data structure of the transaction in Bitcoin.



Each transaction includes multiple transaction inputs and multiple transaction outputs, indicating that one transaction can merge the bitcoins in the previous plurality of accounts and transfer them to another plurality of accounts. Each transaction input is mainly composed of the hash value of the last transaction (PrevTxHash), the output index (Index) and the input script (ScriptSig). The input of a transaction corresponds to the output of the previous transaction, and PrevTxHash saves the hash of the previous transaction output. Index indicates that this transaction input corresponds to which transaction output in the transaction history. The script (ScriptSig) contains the signature of the Bitcoin holder on

the current transaction. Each transaction output includes the transfer amount (Value) and script script-PubKey containing the recipient's public key hash (script-PubKey).

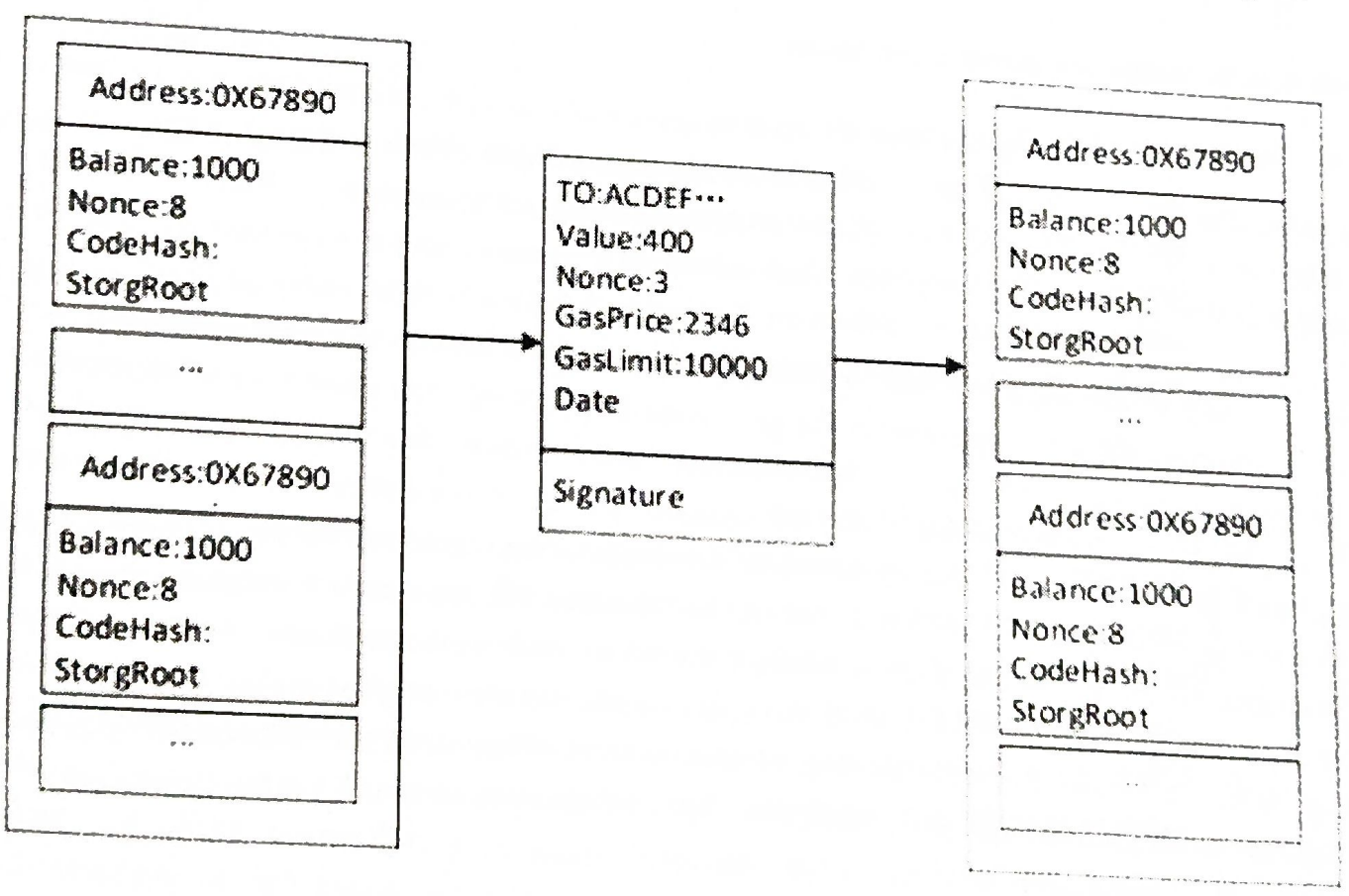
In the transaction-based model, all transactions depend on the hash pointer of the previous transaction to form a list of transactions as nodes. Each transaction can be traced back to the first transaction in the trading chain Coinbase (ie bitcoin from mining). Transaction-based model can effectively prevent counterfeiting, double-flower attacks against digital currency.

2. Account-based model. The transaction-based model makes it easy to verify transactions, but it cannot quickly query user balances. To support more types of industry applications, blockchain platforms such as Ethereum and HyperledgerFabric use an account-based model to easily query transaction balances or trading status data. Smart contracts are also better suited to build on an account-based model that is easier to handle complex business logic for state data.

The accounts under Ethereum are divided into two types:

- External account (Externally Owned Account) and
- Contract account (Contract Account).

The external account is used to record the Ethereum balance, and the contract account is used to store the smart contract. As shown in Figure below, while a transaction occurs, it will trigger a change in account status.



External accounts and contract accounts are represented by the same data structure in Ethereum, which contains four attributes: Balance, Nonce, CodeHash and StorageRoot.

- *Balance* records the Ethereum balance of the account.
- *Nonce* counts the number of transactions initiated by the account to prevent replay attacks.
- *CodeHash* is the hash value of the contract code when the account is applied to a smart contract.
- *StorageRoot* is the MerklePatricia root of contract status data.

Ethereum's transactions include seven attributes: To, Value, Nonce, gasPrice, gasLimit, Data, and transaction signature.

- *To* saves the recipient's account address,
- *Value* is the amount of ethernic currency transferred.
- *Nonce* counts the number of transactions initiated by the transaction originator.
- *GasPrice* is the unit price of the Ether of Gas at the time of the transaction.
- *GasLimit* is the maximum amount of Gas allowed to execute this transaction.
- *Data* is the message data when the smart contract is invoked.
- *Transaction signature* is the initiator's ECDSA signature of the transaction.

Merits of both models are summarized below.

- The benefits of the Transaction Model are:
  - Scalability — Since it is possible to process multiple UTXOs at the same time, it enables parallel transactions and encourages scalability innovation.
  - Privacy —UTXO provides a higher level of privacy, as long as the users use new addresses for each transaction. For enhanced privacy, more complex schemes, such as ring signatures, can be considered.

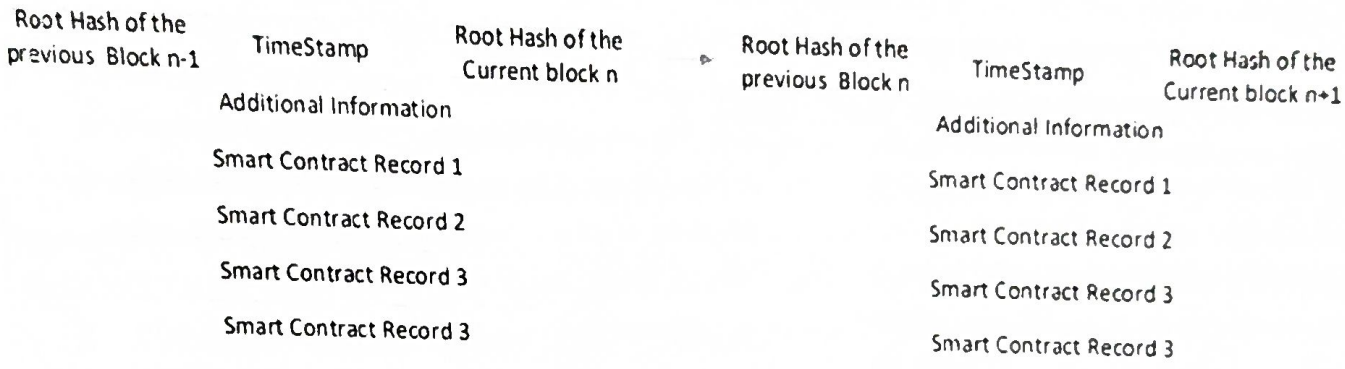
- The benefits of the Account Model are:
  - Simplicity — Ethereum opted for a more intuitive model for the benefit of developers of complex smart contracts, especially those that require state information or involve multiple parties. UTXO's stateless model would force transactions to include state information, and this unnecessarily complicates the design of the contracts.
  - Efficiency — Account/Balance Model is more efficient, as each transaction only needs to validate that the sending account has enough balance to pay for the transaction.

Contract data

Smart contracts stimulate contract execution by using protocols and user interfaces. Smart contract is a commercial contract written in a programming language that automatically enforces the terms of the contract when the predetermined conditions are met, achieving the goal of "code is the law." The existing smart contract works like the 'If-Then' statement of other computer programs. Because blockchain data has important features such as non-tampering and complete credibility, building smart contracts based on blockchain data has natural advantages.

The life cycle of smart contract includes three stages:

1. Contract generation,
2. Contract publication and
3. Contract execution.



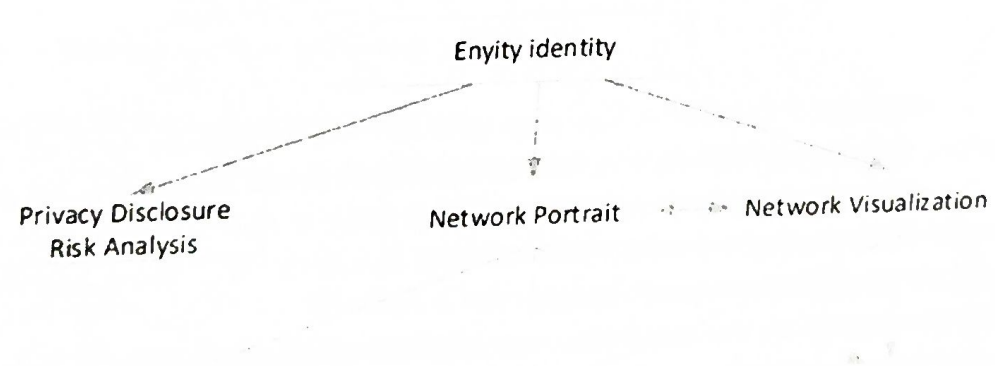
- *Contract generation* mainly includes four aspects: multi-party negotiation, contract specification, contract verification, and contract code. The specific implementation process is as follows: the contract participants negotiate with each other; the parties clarify their respective rights and obligations, jointly determine the contract text, write the program, test the program, and finally obtain the standard contract code.
- *Contract publication* and release is similar to transaction release. The signed contract is distributed to each node through P2P, and each node will temporarily store the received contract in memory and wait for consensus. Each node will package the temporary contract in the most recent period into a contract set, calculate the hash value of the set, and finally assemble the hash value of the contract set into a block and spread to other nodes of the whole network. The node that receives the block will compare the Hash value stored in the block with the Hash value of the contract set saved by itself. Through multiple rounds of transmission and comparison, all nodes will eventually reach a consensus on the newly issued contracts, and the agreed contracts will be distributed to all nodes in the network in block form. As shown in Figure above, Each of the blocks contains the following information: Root Hash of the Current Block, Root Hash of the Previous Block, TimeStamp, contract data, and other descriptive information.
- *Contract execution* is based on the "event-triggered" mechanism. The smart contract on the blockchain contains transaction processing and saving mechanisms and a complete state

machine for accepting and processing various smart contracts. The smart contract periodically traverses the state machine and trigger conditions of each contract, pushing the contract that meets the trigger condition to the queue to be verified. The contract to be verified will spread to each node, just like a normal transaction. The node will first verify the signature to ensure the legitimacy of the contract. The validated contract will be successfully executed after reaching a consensus. Finally, all nodes will reach a consensus on the contract.

The processing of the entire contract is automatically completed by the smart contract system built into the bottom of the blockchain, which is open and transparent and cannot be tampered with. Smart contract system can actively or passively accept, store, process and send data, and call smart contract, so as to control and manage digital objects in the chain. The smart contract technology platforms, have Turing's complete development scripting language, enabling them to support smarter contract applications for financial and social systems.

Problems to be solved in Blockchain data analysis

Based on relevant literature analysis, the focus of blockchain data analysis can be summarized into seven aspects: entity identification, privacy risk analysis, network portrait, network visualization, transaction pattern recognition, market effect analysis, illegal behaviour detection and analysis. Figure below shows the relationship between these seven questions.



Entity identification: In Bitcoin transactions, users are anonymous. Here the transaction involves multiple users, and one user may participate in multiple transactions at the same time. So a question arises: *can you identify users from transaction records, or which addresses belong to the same user?* Since it is not possible to confirm a specific user is identified, it is generally considered that an entity is identified. An entity may be a user or an organization, and these user/organization may also control multiple entities at the same time. Heuristic methods are often used to identify potential entities, which can be divided into two main types:

- Common input method means that all input addresses belong to the same entity in a transaction.
- Change address method uses address definition as a plurality of output addresses of a transaction, and the change address represents an entity.

Privacy protection: can be divided into identity privacy protection and transaction privacy protection.

- Identity privacy protection requires that user's identity information, physical address, IP address are not related to user's public key, address and other public information on the blockchain. Any unauthorized node cannot obtain any information about the user's identity by relying on the data disclosed in the Blockchain. It cannot track and analyse user transactions and identity information through measures such as network monitoring and traffic analysis.
- Transaction privacy protection entails that the data information of the transaction stays anonymous to the unauthorized node. Information like transaction amount, the sender's public key, address of the recipient, and the purchase content of the transaction etc are preserved. Unauthorized nodes cannot obtain these transaction-related knowledge through any effective technical means.

Network portrait: In a massive transaction data environment, we may have to analyse:

- How many users are involved in the transaction?
- What are the characteristics of these users?
- Does this network have the characteristics of a general complex network?
- Bitcoin as an "asset", how is it distributed among users?
- Do you meet the general laws of economics, etc.?

Network Visualization. The transaction data stored in the Blockchain is increasing rapidly. In this rapidly growing trading network, it is quite important to study its visualization tools. For example, the bitcoin trading network's visualization system, BitConeView, can be used to track bitcoin transactions in real time and intuitively. The system defines the concept of "purity" so as to find out the mixed currency transactions conveniently and monitor the real potential money in the Bitcoin network.

Market Effect Analysis. The prices of encrypted currencies are highly volatile. This has attracted economists to discuss features of bitcoin from the perspective of finance. Meanwhile DataAnalysts and BlockChain Researchers focuses on the driving factors behind this extreme volatility. The major influencing factors include:

- Miner behaviour
- Cryptographic currency system settings
- User participation

- Policy
- Number of potential users and market sentiment
- Competition or alternative relationship with other crypto currencies and traditional assets

Illegal behavior detection. Bitcoin is an anonymous, non-centralized payment system. Anonymous features will lead to many illegal activities, such as money laundering, fraudulent gambling, the sale of contraband. and many such illegal acts which are not even known or not even predictable. Hence, Identifying and dealing with these illegal behaviors based on transaction patterns and blockchain data analysis techniques is major area to be focused to preserve the healthy development of blockchain technology. This is a necessity to provide references for legislation of the blockchain industry.

Transaction pattern recognition. Unlike traditional payment systems such as Banks, bitcoin is an anonymous, decentralized payment system.

One area of focus can be, *What characterizes human payment behaviour in the context of anonymity?.*

Another area of focus is, *Whether specific patterns can be identified from blockchain transaction records* to detect related illicit activity.